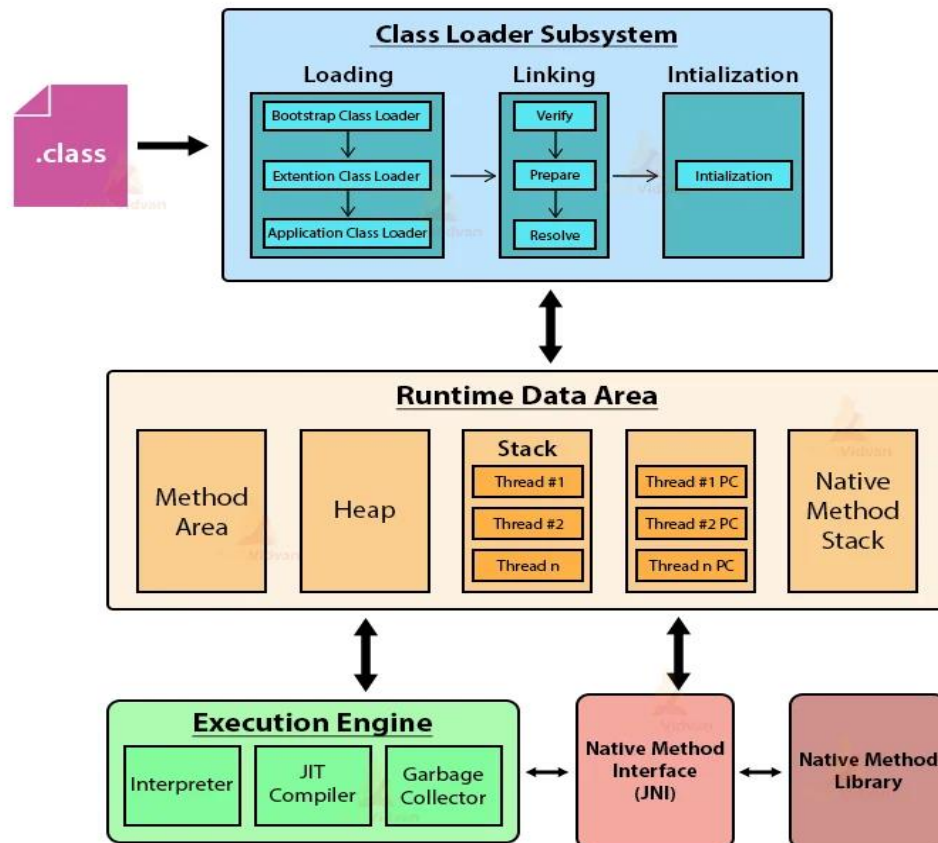


Core Java Questions

JVM Architecture

JVM Model



1. Class Loader Subsystem

Purpose: Loads .class files into JVM memory.

Phases of Class Loading

Phase	Description
Loading	Reads bytecode and creates class object in memory
Linking	Prepares class for execution
Initialization	Assigns actual values to static variables

Types of Class Loaders

Class Loader	Loads
Bootstrap ClassLoader	Core Java classes (java.lang, Object, String)
Extension ClassLoader	Java extension libraries
Application ClassLoader	Your project/application classes

Linking Steps

Step	What Happens
Verify	Checks bytecode safety and correctness
Prepare	Memory allocated for static variables (default values like 0, null)
Resolve	Symbolic references → actual memory references

2. Runtime Data Areas (Memory Structure)

This is where JVM stores data while program runs.

Area	Stores	Shared or Thread?
Method Area	Class metadata, methods, static variables, runtime constant pool	Shared
Heap	Objects and instance variables	Shared
Stack	Method calls, local variables, partial results	Per Thread
PC Register	Address of current executing instruction	Per Thread
Native Method Stack	Native (C/C++) method data	Per Thread

3. Execution Engine

Executes the bytecode.

Component	Role
Interpreter	Executes bytecode line by line
JIT Compiler	Converts frequently used bytecode into native machine code (faster)
Garbage Collector (GC)	Removes unused objects from heap memory

4. JNI (Java Native Interface)

- Allows Java to interact with **native libraries** (C/C++)
 - Used when Java needs OS-level features
-

5. Native Method Libraries

- External libraries written in C/C++
 - Used through JNI
-

Q. New services or features introduced in Java 24

Java 24 mainly focuses on performance improvements, JVM optimizations, garbage collection enhancements, and continued improvements from ongoing projects like Project Loom and Project Panama. It also includes security updates and API refinements to improve stability and developer productivity.

Q How Java memory management works internally

Java memory management is handled automatically by the JVM. Memory is divided into heap, stack, metaspace, and native memory.

Objects are created in the heap, while method calls and local variables are stored in the stack.

The heap is further divided into young and old generations to optimize garbage collection.

The garbage collector automatically removes unused objects based on reachability, reducing memory leaks and manual memory handling.

Q What are primitive types and reference types in Java?

Primitive types in Java are basic data types that store simple values directly in memory. They are not objects and hold their actual values. Java has eight primitive types: byte, short, int, long, float, double, char, and boolean. These types are stored in stack memory (for local variables) and are faster in performance.

Reference types store the memory address (reference) of objects rather than the actual data. Examples include classes, interfaces, arrays, strings, and enums. Reference types are created using the new keyword (except String literals) and are stored in heap memory, while their references are stored in the stack. They can have methods and can hold null values

Q Co-variant return types

Covariant return type means that when a subclass overrides a method of its parent class, the return type of the overridden method can be a subclass of the original return type. This feature was introduced in Java 5 and works only with reference types, not with primitive types. Covariant return types improve readability, flexibility, and reduce the need for explicit type casting while maintaining method overriding rules.

```
class Animal {
    Animal getAnimal() {
        return new Animal();
    }
}

class Dog extends Animal {
    @Override
    Dog getAnimal() {
        return new Dog();
    }
}
```

Q Agile working model and sprint duration

Yes, I work in an Agile environment. Agile focuses on iterative development, continuous feedback, and incremental delivery. Sprint duration is usually two weeks, which includes planning, development, testing, review, and retrospective meetings.

Q Java Generics

Java Generics provide compile-time type safety by allowing classes, methods, and interfaces to operate on typed objects. They reduce runtime `ClassCastException` errors and improve code readability. Generics are widely used in collections such as `List<String>` and `Map<Integer, String>`.

```
class Box<T> {
    private T value;

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) {
        Box<String> box = new Box<>();
        box.set("Hello Java");
        String data = box.get();

        Box<Integer> intBox = new Box<>();
        intBox.set(10);
        int number = intBox.get();
    }
}
```

Q Difference between ClassNotFoundException and NoClassDefFoundError

ClassNotFoundException occurs when an application tries to load a class dynamically at runtime but the class is not found in the classpath. It is a checked exception.

NoClassDefFoundError occurs when a class was present during compilation but is missing at runtime, usually due to missing dependencies. It is an unchecked error.

Q What is cache and its types in frontend and backend

Cache is a temporary storage used to store frequently accessed data for faster retrieval. In the frontend, caching includes browser cache, local storage, and CDN cache to reduce network latency.

In the backend, caching includes in-memory caches like Redis or Ehcache, database-level cache, and application-level cache to reduce database load and improve performance.

Q Custom exception creation and usage in real projects

Custom exceptions are created by extending Exception or RuntimeException. They are used to represent application-specific or business logic errors such as invalid input or unauthorized access. Custom exceptions improve code clarity and make error handling more meaningful and maintainable.

Q Custom sorting using Comparable and Comparator

Comparable is used to define the natural ordering of objects by implementing the compareTo() method inside the class.

Using Comparable (natural sorting order)

Here the object is always sorted by **id** because the sorting logic is inside the class.

```
java Copy code

class Employee implements Comparable<Employee> {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Employee e) {
        return this.id - e.id; // sort by id
    }
}
```

When `Collections.sort(employeeList)` is called, employees are sorted by **id** automatically.

Comparator is used to define external or multiple sorting strategies by implementing the `compare()` method. Comparator is preferred when multiple sorting logics are required.

Using Comparator (multiple sorting logic)

Here sorting logic is external and can change based on requirement.

```
java

import java.util.Comparator;

class SortByName implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.name.compareTo(e2.name); // sort by name
    }
}
```

Usage:

```
java

Collections.sort(employeeList, new SortByName());
```

Q How to detect and fix memory leaks in Java applications

Memory leaks can be detected by monitoring heap usage and observing continuous memory growth. Heap dumps can be taken and analyzed using tools like JVisualVM or Eclipse MAT to identify unreachable objects still referenced. Common causes include static references, unclosed

resources, and improper collection handling. Fixing involves releasing references and proper resource management.

Q What is ExecutorService vs ForkJoinPool?

ExecutorService is a framework provided by Java to manage and execute asynchronous tasks using a pool of threads. It is mainly used for independent tasks such as handling requests, background jobs, or I/O operations. ExecutorService uses fixed, cached, or scheduled thread pools and provides better control over thread lifecycle compared to manually creating threads.

ForkJoinPool is a specialized implementation of ExecutorService designed for parallel and recursive tasks. It works on the divide-and-conquer approach where a large task is split into smaller subtasks, processed in parallel, and then combined. ForkJoinPool uses a work-stealing algorithm to maximize CPU utilization and is best suited for CPU-intensive operations such as parallel streams and complex computations.

In summary, ExecutorService is used for general-purpose task execution, while ForkJoinPool is optimized for parallel processing and recursive workloads.

Q Immutability and how to create immutable classes

Immutability means an object's state cannot be changed after creation. An immutable class is created by making the class final, fields private and final, and not providing setter methods. Immutable objects are thread-safe and easier to maintain, with String being a common example.

What are the major features introduced in Java 8?

Java 8 introduced lambda expressions, functional interfaces, Stream API, default and static methods in interfaces, Optional class, new Date and Time API, method references, Nashorn JavaScript engine, and

improvements in Collections and concurrency. These features mainly support functional programming and improved code readability.

Q What are lambda expressions in Java 8?

Lambda expressions provide a concise way to represent anonymous functions. They allow passing behavior as a parameter and reduce boilerplate code. Lambda expressions are mainly used with functional interfaces and help in writing cleaner and more readable code.

Q What is a functional interface?

A functional interface is an interface that contains exactly one abstract method. It can have multiple default or static methods. Functional interfaces are used as the target type for lambda expressions. Examples include Runnable, Callable, and Comparator.

What is the BiConsumer interface?

BiConsumer is a functional interface from the `java.util.function` package that accepts two input arguments and returns no result. It is mainly used when an operation needs to consume two values, such as processing key-value pairs in a Map using the `forEach` method.

What is the @FunctionalInterface annotation?

The `@FunctionalInterface` annotation is used to indicate that an interface is intended to have exactly one abstract method. It helps the compiler enforce this rule and throws an error if more than one abstract method is added. It also improves code readability and clarity for developers.

Can you name a few functional interfaces?

Common functional interfaces in Java include Runnable, Callable, Comparator, Consumer, Supplier, Predicate, Function, BiConsumer, BiFunction, and UnaryOperator.

Q Difference between Collection and Stream?

A Collection stores data in memory, while a Stream processes data. Collections are reusable and mutable, whereas streams are single-use and focus on computation. Streams support functional-style operations and lazy evaluation.

Q Can an abstract class have a constructor?

Yes, an abstract class can have a constructor in Java. The constructor is used to initialize common fields of the abstract class and is called automatically when a subclass object is created. Even though an abstract class cannot be instantiated directly, its constructor executes as part of the subclass object creation process.

Q What are static methods in interfaces?

Static methods in interfaces belong to the interface itself and cannot be overridden by implementing classes. They are used for utility or helper methods related to the interface.

Q What is the Optional class?

Optional is a container object used to represent the presence or absence of a value. It helps avoid NullPointerException by providing methods to handle null values safely and encourages better null handling practices.

Q What is the new Date and Time API?

Java 8 introduced a new Date and Time API under the java.time package. It provides immutable and thread-safe classes like LocalDate, LocalTime, and LocalDateTime. This API resolves issues present in the old Date and Calendar classes.

Q What are method references?

Method references provide a shorthand way to refer to methods using the :: operator. They improve code readability and are mainly used with lambda expressions. Method references can refer to static methods, instance methods, or constructors.

```
import java.util.Arrays;
import java.util.List;

public class MethodReferenceExample {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", "Spring", "Microservices");

        // Using Lambda expression
        names.forEach(name -> System.out.println(name));

        // Using method reference
        names.forEach(System.out::println);
    }
}
```



Q What improvements were made to Collections in Java 8?

Java 8 added new methods like `forEach`, `removeIf`, `replaceAll`, and `computeIfAbsent`. These methods simplify collection processing and support functional programming using lambda expressions.

Q What concurrency enhancements were introduced in Java 8?

Java 8 improved concurrency by enhancing the `ForkJoinPool`, introducing parallel streams, and adding new methods in concurrent collections. These changes improve performance and scalability in multi-threaded applications.

Q Why is Java 8 considered a major release?

Java 8 is considered a major release because it introduced functional programming concepts into Java. It significantly reduced boilerplate code,

improved performance, enhanced APIs, and changed the way Java applications are written and maintained.

Q What are modern Java features like Records, GC updates, var keyword, modules, and sealed classes?

Modern Java versions focus on reducing boilerplate, improving performance, and increasing safety.

The var keyword allows local variable type inference, meaning the compiler automatically determines the variable type based on the assigned value.

Records provide a concise way to create immutable data carriers.

The module system (introduced in Java 9) improves application structure and dependency management.

Sealed classes restrict which classes can extend or implement them, enhancing design control. Garbage Collector updates such as G1 and ZGC improve performance, scalability, and low-latency behavior.

Q Method overriding vs creating a new method

Method overriding occurs when a subclass provides its own implementation of a method already defined in the parent class with the same method signature. It supports runtime polymorphism. Creating a new method means defining a method with a different signature, which does not affect the parent class behavior and does not participate in polymorphism.

Q Local and global exception handling

Local exception handling is done using try-catch blocks within a method to handle errors immediately where they occur. Global exception handling is used to handle exceptions at a centralized level, such as using a global

exception handler in frameworks like Spring, ensuring consistent error responses and cleaner business logic.

Q What is the Record class and where is it used?

A Record is a special class introduced to represent immutable data objects. It automatically generates constructors, getters, equals, hashCode, and toString methods. Records are commonly used for DTOs, API responses, and data-centric models.

Note:- the record class is immutable that's why it won't provide setter methods

```
record Person(String name, int age) {}

public class Demo {
    public static void main(String[] args) {

        Person p = new Person("Ram", 25);

        System.out.println(p.name()); // Ram
        System.out.println(p.age());  // 25
    }
}
```

How does Record improve DTO handling in Java?

Records reduce boilerplate code by automatically generating commonly used methods. They enforce immutability, improve readability, and make DTO definitions concise and less error-prone, which is especially useful in microservices and API-based applications.

Difference between Optional.get(), orElse(), and orElseThrow()

Optional.get() returns the value if present but throws an exception if empty. orElse() returns a default value when the Optional is empty. orElseThrow() throws a custom or default exception when no value is present, making error handling explicit.

What are Default Methods?

Default methods allow interfaces to contain method implementations using the default keyword. They were introduced to maintain backward compatibility when enhancing existing interfaces without breaking implementations.

What is HTTP Status Code 204?

HTTP status code 204 means “No Content.” It indicates that the request was successfully processed, but there is no response body to return. It is commonly used for successful delete or update operations.

What is Method Hiding?

Method hiding occurs when a static method in a subclass has the same signature as a static method in the parent class. Static methods are resolved at compile time, so method hiding does not support runtime polymorphism.

Output of: `System.out.println(Double.MIN_VALUE > 0.0d);`

The output is true because Double.MIN_VALUE represents the smallest positive non-zero value, not the most negative value.

If you add a null value to an empty Set, what will be the size of the Set?

The size of the Set will be 1 because most Set implementations like HashSet allow a single null element.

Q Can you override a static method in an interface?

No, static methods in interfaces cannot be overridden. They belong to the interface itself and can only be accessed using the interface name.

What are switch expressions and how are they different from switch statements?

Switch expressions were introduced to make switch more concise and expressive. Unlike traditional switch statements, switch expressions return a value and can be used directly in assignments. They use the `->` syntax and do not require break statements, reducing fall-through errors. Switch expressions also support multiple labels and are easier to read and maintain.

What is the SOLID Principle?

The SOLID principle is a set of five object-oriented design principles aimed at making software more maintainable and scalable. It includes Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. These principles help in writing clean, flexible, and testable code.

Difference between `==` and `equals()`

The `==` operator compares references, meaning it checks whether two variables point to the same memory location. The `equals()` method compares the actual content or logical equality of objects. For primitive types, `==` compares values, but for objects, `equals()` should be used for meaningful comparison.

Optional Class (Java)

The Optional class is a container object used to **avoid null values** and prevent `NullPointerException`. It represents a value that may or may not be present. Instead of returning null, methods can return an Optional to make the absence of a value explicit and safer to handle.

```
Optional<String> name = Optional.ofNullable(null);

// Using default value if empty
String result = name.orElse("Default Name");

System.out.println(result); // Output: Default Name
```

Sealed Classes (Java)

A **sealed class** is used to **restrict which classes can extend or implement it**. It provides controlled inheritance, improving code safety and design.

```
sealed class Animal permits Dog, Cat {}

final class Dog extends Animal {}

non-sealed class Cat extends Animal {}

class PersianCat extends Cat {} // allowed because Cat is non-sealed
```

HASHMAP

What is the Map interface in Java?

The Map interface represents a collection of key–value pairs where each key is unique. It does not allow duplicate keys but allows duplicate values. Map is mainly used for fast lookup, insertion, and deletion based on keys. Common implementations include HashMap, LinkedHashMap, TreeMap, Hashtable, and ConcurrentHashMap.

What is HashMap?

HashMap is the most commonly used implementation of the Map interface. It stores data in key–value pairs using hashing for fast access. HashMap allows one null key and multiple null values. It is not thread-safe,

so it should not be used directly in multi-threaded environments without synchronization.

What is ConcurrentHashMap?

ConcurrentHashMap is a thread-safe Map implementation designed for high concurrency. It allows multiple threads to read and write simultaneously without locking the entire map. It does not allow null keys or null values. It is commonly used in multi-threaded applications where performance and scalability are important.

What is SynchronizedMap?

SynchronizedMap is a thread-safe wrapper created using `Collections.synchronizedMap()`. It synchronizes the entire map, allowing only one thread to access it at a time. While it ensures thread safety, it can cause performance issues in highly concurrent environments due to full-map locking.

What is LinkedHashMap?

LinkedHashMap is an extension of HashMap that maintains the insertion order or access order of elements. It allows one null key and multiple null values. LinkedHashMap is commonly used when predictable iteration order is required, such as implementing LRU caches.

What is Hashtable?

Hashtable is a legacy Map implementation that is thread-safe by default. It synchronizes every method, which makes it slower compared to modern alternatives. Hashtable does not allow null keys or null values and is rarely used in new applications.

What is SortedMap?

SortedMap is a sub-interface of Map that maintains keys in a sorted order,

either natural ordering or using a custom comparator. It provides methods to access a range of keys such as `firstKey`, `lastKey`, `headMap`, and `tailMap`.

What is TreeMap?

`TreeMap` is an implementation of the `SortedMap` interface that stores keys in sorted order. It does not allow null keys but allows multiple null values. `TreeMap` is commonly used when sorted data or range-based operations are required.

Difference between HashMap, ConcurrentHashMap, and SynchronizedMap with real-time usage

`HashMap` is used in single-threaded or read-only scenarios where performance is critical, such as caching configuration data.

`ConcurrentHashMap` is used in multi-threaded applications like web servers where multiple threads need to access and update shared data concurrently. `SynchronizedMap` is used in simpler multi-threaded scenarios where concurrency is low and ease of implementation is preferred over performance.

HashMap and concurrency

`HashMap` is not thread-safe. When multiple threads modify a `HashMap` concurrently without proper synchronization, it can lead to data inconsistency, infinite loops during resizing, and unpredictable behaviour. Therefore, `HashMap` should be used only in single-threaded scenarios or when external synchronization is applied.

What is the contract between hashCode() and equals()?

The contract states that if two objects are equal according to the `equals()` method, they must return the same `hashCode()`. However, if two objects have the same `hashCode()`, they are not necessarily equal. This contract is essential for correct behavior of hash-based collections like `HashMap` and `HashSet`.

What enhancement was made in HashMap after Java 1.8?

After Java 8, HashMap introduced the use of Red-Black Trees in buckets to improve performance in case of high hash collisions. When too many entries are stored in a single bucket, lookup time improves from $O(n)$ to $O(\log n)$. Internal hashing and resizing mechanisms were also optimized.

After which point does HashMap convert a bucket into a Red-Black Tree?

A HashMap bucket is converted into a Red-Black Tree when the number of entries in a single bucket exceeds 8 and the overall capacity of the HashMap is at least 64. If the number of entries later drops below 6, the tree is converted back to a linked list.

How does locking work in ConcurrentHashMap?

In Java 8 and later, ConcurrentHashMap does not lock the entire map. It uses a combination of CAS (Compare-And-Swap) operations and synchronized blocks at the bucket level. This fine-grained locking allows multiple threads to operate on different buckets simultaneously, improving concurrency and performance.

What is the default bucket size of HashMap?

The default initial capacity (bucket size) of a HashMap is 16. This capacity grows dynamically as more elements are added based on the load factor.

With bucket size 16, how many read/write operations can happen simultaneously?

In older versions of how streams work internally (before Java 8), the default concurrency level was 16, meaning up to 16 write operations could occur simultaneously. In Java 8 and later, there are no segments, and

concurrency is not directly tied to bucket size; instead, multiple threads can operate concurrently on different buckets depending on contention.

Why does `HashMap.containsValue()` have $O(n)$ time complexity?

`HashMap.containsValue()` checks whether a given value exists in the map by iterating over all entries. `HashMap` is optimized for key-based lookup using hashing, not for value-based searches. Since values are not stored in a hashed or indexed structure, the method must scan each entry until the value is found or all entries are checked, resulting in $O(n)$ time complexity.

`final HashMap<Employee> emp = new HashMap<>();` — what can and can't change

The `final` keyword means the reference `emp` cannot be reassigned to another `HashMap`. However, the contents of the map can still be modified. You can add, remove, or update entries, but you cannot point `emp` to a new `HashMap` instance.

`HashMap` internal working, collision handling, and `HashMap` vs `LinkedHashMap`

`HashMap` stores key-value pairs in an array of buckets using the hash code of the key. When multiple keys map to the same bucket, a collision occurs. Collisions are handled using a linked list or, after Java 8, a Red-Black Tree when the bucket size exceeds a threshold. `LinkedHashMap` extends `HashMap` and maintains insertion or access order using a doubly linked list, making iteration predictable but slightly slower and more memory-intensive than `HashMap`.

Scenario: how `HashMap` behaves when keys are mutable

If a mutable object is used as a key and its fields affecting `hashCode()` or `equals()` are changed after insertion, `HashMap` may fail to locate the entry. The key may be placed in the wrong bucket, causing `get()` or `containsKey()` to return null. This is why keys in `HashMap` should always be immutable.

How HashMap works internally including resizing mechanism

HashMap uses an internal array where each index is called a bucket. The bucket index is calculated using the hash code of the key. When the number of entries exceeds the threshold (capacity $\times$ load factor, default 0.75), the map resizes by doubling its capacity. During resizing, all existing entries are rehashed and redistributed into the new bucket array, which is an expensive operation.

What is load factor and why is the default value 0.75?

The load factor defines how full a HashMap can get before it resizes. It is the ratio of the number of entries to the number of buckets. The default value of 0.75 provides a good balance between memory usage and performance. A lower load factor reduces collisions but increases memory usage, while a higher load factor saves memory but increases collision probability. The value 0.75 is chosen as an optimal trade-off for most use cases.

Why does HashMap allow one null key?

HashMap allows one null key because it treats the null key as a special case and always stores it in a fixed bucket. Since there can be only one unique key, only one null key is allowed. This design simplifies usage and provides backward compatibility, while still maintaining correct map behaviour.

Difference between put() and putIfAbsent()

The put() method inserts the key-value pair into the map and overwrites the existing value if the key already exists. The putIfAbsent() method inserts the key-value pair only if the key is not already present or is mapped to null. putIfAbsent() is commonly used in concurrent environments to avoid accidental overwriting of existing values.

Why does ConcurrentHashMap disallow null keys and null values?

ConcurrentHashMap disallows null keys and values to avoid ambiguity during concurrent operations. In a multi-threaded environment, a null return value could mean either that the key is absent or that the value associated with the key is null. By disallowing nulls, ConcurrentHashMap ensures clear and predictable behavior, especially for methods like `get()` and `containsKey()`.

Multi Threading

What happens inside the JVM when a thread is created?

When a thread is created in Java, the JVM allocates memory for the thread object and creates a separate call stack for it. The thread is registered with the JVM thread scheduler but does not start execution immediately. The thread remains in the NEW state until the `start()` method is called, after which the JVM schedules it for execution based on CPU availability.

What happens if two threads try to update the same record?

When two threads try to update the same record simultaneously, a race condition can occur, leading to inconsistent or corrupted data. To prevent this, synchronization mechanisms are used. Pessimistic locking blocks other threads until the current thread finishes the update, while optimistic locking allows concurrent access but checks for conflicts before committing changes, commonly using versioning.

Difference between lock and synchronization

Synchronization is a language-level mechanism using the `synchronized` keyword and is managed by the JVM automatically. Lock is provided by the `java.util.concurrent.locks` package and offers more control, such as `try-`

locking, fairness policies, and interruptible locking. Locks are more flexible and suitable for complex concurrency scenarios.

How does a lock work in Java?

A lock works by controlling access to a shared resource. When a thread acquires a lock, other threads attempting to acquire the same lock are blocked until it is released. Locks allow explicit lock and unlock operations, giving developers better control over thread coordination and execution flow.

All methods of the Thread class (important ones)

The Thread class provides methods such as `start()` to begin execution, `run()` to define task logic, `sleep()` to pause execution, `join()` to wait for another thread to finish, `yield()` to give execution chance to other threads, `interrupt()` to signal interruption, `isAlive()` to check thread state, and `getName()` or `setName()` to manage thread identity.

What is ReentrantLock and its methods?

ReentrantLock is a lock implementation that allows a thread to acquire the same lock multiple times without causing a deadlock. It provides methods such as `lock()`, `unlock()`, `tryLock()`, `lockInterruptibly()`, and `newCondition()`. It offers better flexibility than synchronized blocks, including fairness and interrupt handling.

What is context switching?

Context switching is the process where the CPU switches execution from one thread to another. During this process, the current thread's state is saved and the next thread's state is restored. Context switching allows multitasking but introduces overhead, which can affect performance if excessive.

What is thread lifecycle and difference between start() and run()?

A thread lifecycle includes states such as NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. Calling start() creates a new thread and invokes the run() method internally, enabling concurrent execution. Calling run() directly does not create a new thread; it executes the method like a normal function in the current thread.

Try-With-Resources (Java)

Try-with-resources is used to automatically close resources (like files, streams, database connections) after use. It helps avoid memory leaks and removes the need for a finally block.

```
import java.io.FileInputStream;

public class Demo {
    public static void main(String[] args) {

        try (FileInputStream fis = new FileInputStream("file.txt")) {
            System.out.println("Reading file...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

What are virtual threads in Java 21 and how do they differ from traditional threads?

Virtual threads are lightweight threads introduced as part of Project Loom. They are managed by the JVM instead of the operating system and can scale to millions of concurrent threads with minimal overhead. Unlike traditional platform threads, virtual threads are ideal for I/O-bound tasks and significantly reduce resource consumption while simplifying concurrent programming.

Multithreading Guide:-

<https://engineeringdigest.medium.com/multithreading-in-java-39f34724bbf6>

Stream

Q What is the Stream API?

The Stream API is used to process collections of data in a functional and declarative way. It supports operations like filtering, mapping, and reducing data. Streams do not store data; they process data from a source and support parallel execution for better performance.

How do Java Streams work internally?

Java Streams work on a source such as a collection, array, or I/O channel and process data through a pipeline of operations. The stream pipeline consists of intermediate operations and a terminal operation. Streams use lazy evaluation, meaning intermediate operations are not executed until a terminal operation is called. Internally, elements flow through the pipeline one by one, allowing optimizations like short-circuiting and parallel execution.

What is a stream function?

A stream function refers to operations provided by the Stream API to process data in a functional style. These functions operate on elements of

a stream and include operations like map, filter, reduce, and collect. They are usually implemented using lambda expressions and do not modify the original data source.

From how many sources can a stream be created?

Streams can be created from multiple sources such as collections, arrays, files, I/O channels, generators, and static factory methods like `Stream.of()` or `Stream.iterate()`. Streams can also be created as sequential or parallel streams depending on the requirement.

What are the benefits of using streams and functional programming in Java?

Streams and functional programming make code more readable, concise, and expressive. They reduce boilerplate code, improve maintainability, and support parallel execution with minimal effort. Streams also promote immutability and help avoid side effects, leading to safer and cleaner code.

What are intermediate operations in Streams?

Intermediate operations are operations that transform a stream into another stream. They are lazy and are not executed until a terminal operation is invoked. Examples include map, filter, distinct, sorted, and limit. These operations can be chained together to form a processing pipeline.

What is the difference between map() and filter()?

The `map()` operation is used to transform each element of a stream into another form, such as converting objects or modifying values. The `filter()` operation is used to select elements based on a condition and returns only those elements that satisfy the predicate. Map changes the data, while filter reduces the data.

Difference between map() and flatMap()?

map() transforms each element into a single output element. flatMap() transforms each element into a stream and then flattens all resulting streams into a single stream. flatMap() is commonly used when dealing with nested collections or streams.

What happens if you try to add an element to a list created using stream().toList()? Why?

A list created using stream().toList() is immutable. If you try to add or remove elements, it throws an UnsupportedOperationException. This is because toList() returns an unmodifiable list to encourage immutability and safer functional-style programming.

Sequential Stream vs Parallel Stream

A sequential stream processes elements one by one on a single thread. A parallel stream divides the data into multiple parts and processes them concurrently using the ForkJoinPool. Parallel streams can improve performance for large, CPU-intensive tasks but may degrade performance for small datasets or I/O operations due to thread overhead.

Are Java Streams reusable?

No, Java streams are single-use. Once a terminal operation like collect() or forEach() is executed, the stream is closed. Reusing it will throw an IllegalStateException. This design ensures safe and predictable data processing.

Why are streams lazy?

Streams are lazy because intermediate operations are not executed immediately. They run only when a terminal operation is invoked. Laziness allows streams to optimize execution, reduce unnecessary computation, and enable short-circuiting operations like findFirst() and limit().

Difference between `forEach()` and `forEachOrdered()`

`forEach()` does not guarantee encounter order, especially in parallel streams, and is faster.

`forEachOrdered()` maintains the original order of elements but may reduce performance in parallel execution.

Intermediate vs Terminal Operations

Intermediate operations (`map`, `filter`, `peek`) return a stream and are lazy.

Terminal operations (`collect`, `reduce`, `forEach`) trigger execution and produce a result or side effect. A stream pipeline must end with exactly one terminal operation.

What are short-circuiting operations?

Short-circuiting operations stop stream processing once a condition is satisfied. Examples include `findFirst()`, `anyMatch()`, and `limit()`. They improve performance by avoiding full traversal of the stream.

Difference between `peek()` and `map()`

`map()` transforms elements and returns a new stream.

`peek()` is used mainly for debugging; it performs actions without modifying elements. It should not be used for business logic.

What is a Collector in Java?

A **Collector** is a special object used in Java Streams to **transform the elements of a stream into a final result** such as a `List`, `Set`, `Map`, or even a single computed value.

Collectors are mainly used with the **`collect()` terminal operation**.

collect() vs reduce()

reduce() combines stream elements into a single value using an accumulator.

collect() is a mutable reduction used to transform stream data into collections like List, Set, or Map using collectors.

Difference between Collectors.toList() and Stream.toList()

Collectors.toList() returns a mutable list (implementation not guaranteed).

Stream.toList() (Java 16+) returns an unmodifiable list. Attempting to modify it throws UnsupportedOperationException.

How do parallel streams work internally?

Parallel streams use the ForkJoin framework. The data source is split into smaller chunks using a Spliterator, processed in parallel by worker threads, and then combined. They use the common ForkJoinPool by default.

Why parallel streams can be slower?

Thread creation, task splitting, merging results, synchronization, and cache contention can outweigh benefits for small datasets or simple operations.

Why stateful lambdas are dangerous in streams?

Stateful lambdas modify shared variables, causing race conditions and unpredictable results, especially in parallel streams. Streams are designed for stateless, side-effect-free operations.

Are streams immutable?

Yes, streams do not modify the source collection. They operate on data and produce new results, preserving immutability and reducing side effects.

Can streams modify the original collection?

No, streams cannot directly modify the source collection. Any modification must be done explicitly outside the stream, otherwise it may cause `ConcurrentModificationException`.

How to handle checked exceptions in streams?

Streams do not support checked exceptions directly. Common approaches include wrapping exceptions in runtime exceptions, using helper methods, or handling exceptions inside lambda expressions.

findFirst() vs findAny()

`findFirst()` returns the first element based on encounter order and is deterministic.

`findAny()` may return any element and is optimized for parallel streams, offering better performance.

What is Spliterator?

A `Spliterator` is an internal iterator that splits data sources for parallel processing. It improves performance by enabling efficient data partitioning and traversal.

Why streams are not a replacement for loops?

Streams are ideal for declarative data processing but lack fine-grained control, complex conditional logic, and easy debugging. Loops are better for low-level control and performance-critical code.

Why stream().count() is a terminal operation but size() is not

`count()` is a `Stream` terminal operation. It triggers stream processing and consumes the stream to produce a result (a `long`). After `count()` is called, the stream is closed and cannot be reused.

`size()` belongs to the `Collection` interface, not `Stream`. It simply returns the

current number of elements stored in the collection without processing data in a pipeline, so it is not a terminal operation.

Can a stream be used after collect()?

No, a stream cannot be reused after `collect()` or any terminal operation. Streams are designed for single-use processing. Once a terminal operation is executed, the stream is considered consumed and closed. Attempting to reuse it results in an `IllegalStateException`.

Why modifying a collection while streaming causes `ConcurrentModificationException`

Most collections like `ArrayList` and `HashMap` use fail-fast iterators. When a stream is created, it internally uses an iterator. If the underlying collection is structurally modified (add/remove) while the stream is processing, the iterator detects this change using a modification count and throws `ConcurrentModificationException` to prevent inconsistent results.

Difference between `limit()` and `skip()`

`limit(n)` restricts the stream to the first `n` elements and ignores the rest. `skip(n)` ignores the first `n` elements and processes the remaining ones. Both are short-circuiting intermediate operations and are often used together for pagination logic.

Difference between `distinct()` and `Set`

`distinct()` is a Stream intermediate operation that removes duplicate elements during stream processing while preserving encounter order. It uses `equals()` and `hashCode()` internally.

A `Set` is a data structure that does not allow duplicates and enforces uniqueness at the storage level. Converting a stream to a `Set` removes duplicates but may lose ordering depending on the `Set` implementation.

Serialization / Deserialization

Q What is Serialization?

Serialization in Java is the process of converting a live Java object into a *sequence of bytes* so it can be persisted (saved to a file), sent over a network, or stored for later use. This makes it possible to transfer object state between JVMs or save it to disk. Deserialization is the opposite process — it reconstructs the original object from its byte stream.

Q What is the Serializable Interface?

In Java, a class must implement the `java.io.Serializable` interface if you want its objects to be serializable. **This interface is a *marker interface*, meaning it contains no methods — it simply *marks* the class as capable of serialization.** The JVM checks for this marker at runtime to allow serialization/deserialization.

```
public class Person implements Serializable {  
    private String name;  
    private int age;  
}
```

During serialization, Java uses `ObjectOutputStream.writeObject(obj)` and during deserialization uses `ObjectInputStream.readObject()` to rebuild the object.

Q transient Keyword

The `transient` keyword is a *field modifier* used to *exclude specific fields* from serialization. When an object is serialized, transient fields are ignored; their values are replaced with default values during deserialization (null for

objects, 0 for primitives). This is useful for sensitive data (like passwords) or data you *don't want to persist*.

Why use transient?

- Protects *confidential* or *security-sensitive* information.
- Avoids storing *temporary* or *derived values*.
- Helps reduce *serialized size* if some fields aren't necessary to persist

Q: What happens to a transient field after deserialization?

A: It gets the default value (0, null, false, etc.) because it was not saved in the byte stream.

Q: What is serialVersionUID and why is it important?

A: It's a unique version identifier for Serializable classes. Mismatched UID between serialized and deserialized versions causes `InvalidClassException`. You should declare `serialVersionUID` explicitly to ensure compatibility across versions.

✓ Q: How do you serialize an object to a file in Java?

Answer: You create `FileOutputStream` for a file and wrap it with `ObjectOutputStream`, then call `writeObject()`. During deserialization, use `FileInputStream` and `ObjectInputStream.readObject()`. Ensure the class implements `Serializable`.

Q: Can static fields be serialized?

Answer: No — static fields belong to the class, not object state, so they are not serialized by default.

Q: Why do we use serialization in file handling?

Answer: It allows persisting complex object graphs (objects with fields) to disk in a form that can be restored later, simplifying I/O beyond primitive reads/writes.

Q: What exception might be thrown if a class isn't Serializable?

Answer: `NotSerializableException` occurs when trying to serialize an object whose class does not implement `Serializable`.

Q: Difference between Serializable and Externalizable?

Answer: Serializable uses default mechanisms with optional custom writeObject/readObject methods, whereas Externalizable requires implementing methods to fully control serialization. (Often asked in interviews.)

Java 24 (March 2025)

Java 24 is a *feature release* that includes new enhancements focused on language expressiveness, performance, and experimental features. Key areas include:

- **Pattern Matching for Primitive Types (Preview):** Enhances type-safe pattern matching for primitives in instanceof and switch.
- **Flexible Constructor Bodies (Preview):** Allows statements before super()/this() in constructors to initialize fields earlier.
- **Module Import Declarations:** Simplifies modular dependencies with declarative imports at the top of Java files.
- **Compact Object Headers (Experimental):** Reduces memory footprint for object headers for better performance.
- **Other Enhancements:** Vector API incubator, Scoped Values previews, Structured Concurrency improvements, security enhancements, and more.

Tip: When asked about Java 24 in interviews, focus on *preview language features, performance improvements, and how these help real-world coding.*

java 25 (September 2025 – LTS)

Java 25 is the **latest Long-Term Support (LTS)** release and represents a major update. Interviewers typically ask about key new language features, runtime improvements, and APIs. Major features include:

✨ **Key Language and API Features:**

- **Scoped Values:** Immutable contextual data that can replace ThreadLocal in many use cases.
- **Structured Concurrency (Preview):** Simplifies running related tasks together with better cancellation/error handling.
- **Pattern Matching Enhancements:** Primitive types are now supported in patterns.
- **Module Import Declarations & Flexible Constructors:** Continued language simplification.
- **Performance & JVM Enhancements:**
 - *Compact Object Headers* improves memory efficiency.
 - *Ahead-of-Time (AOT) Method Profiling* boosts warm-up performance.
 - *Garbage Collector improvements (Generational Shenandoah)* enhance pause times.
 - *JFR CPU-Time Profiling* and *Method Timing & Tracing* offer detailed profiling.

Security & Platform Changes:

- Removal of 32-bit x86 support.
- New key derivation function API and cryptographic enhancements.

 *Interview tip:* Emphasize that Java 25 is LTS with practical production usage, performance gains, developer ergonomics, and security improvements.